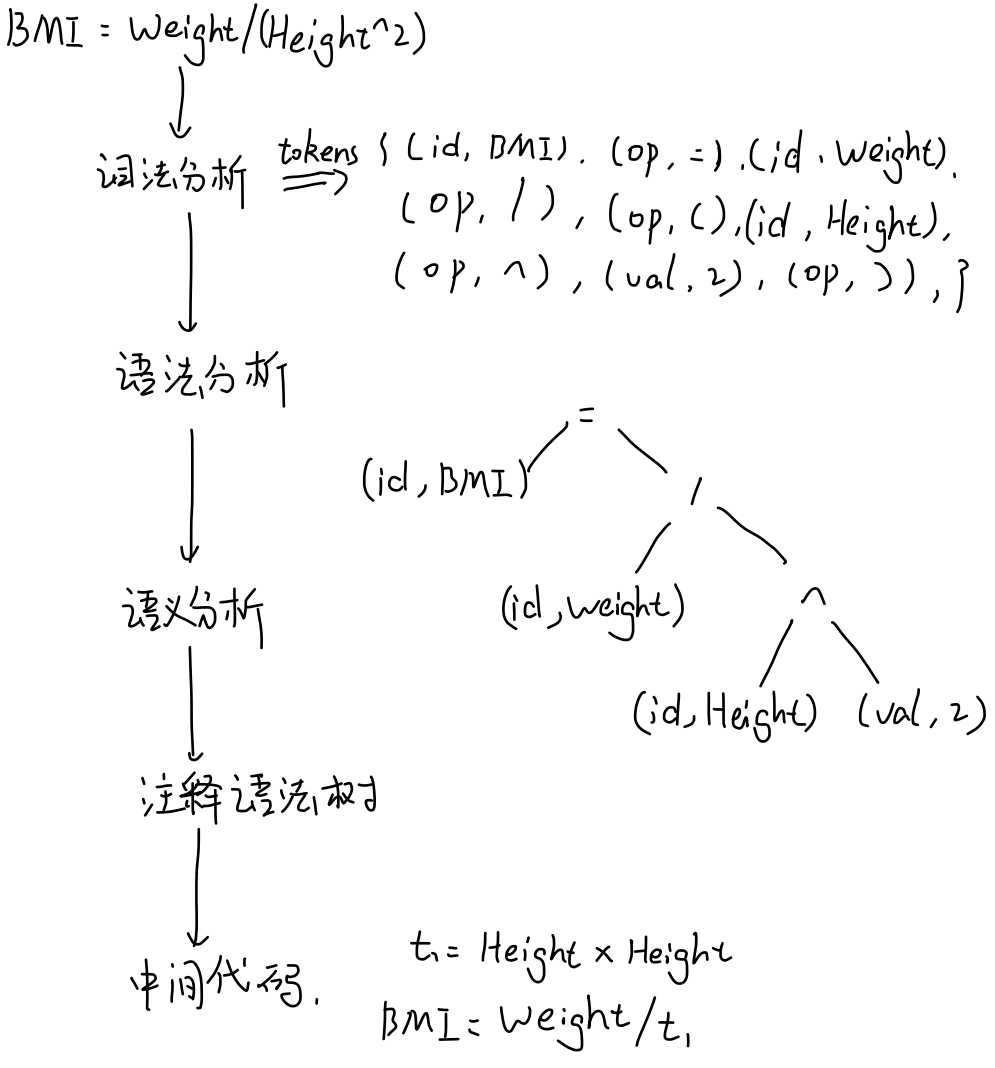
1. Give the output to each stage of the compiler according to the input: "BMI=Weight/(Height^2)"



2. A lexical analyzer implemented based on the lexical specification as below. Please give the output of the lexical analyzer while the input of 'ababcbacaabc'.

$$\sum = \{a, b, c\} \tag{1}$$
$$L_1 = (a|b) * c \tag{2}$$
$$L_2 = ab * a \tag{3}$$
$$L_3 = ab(ab) * |ab(ab) * c \tag{4}$$

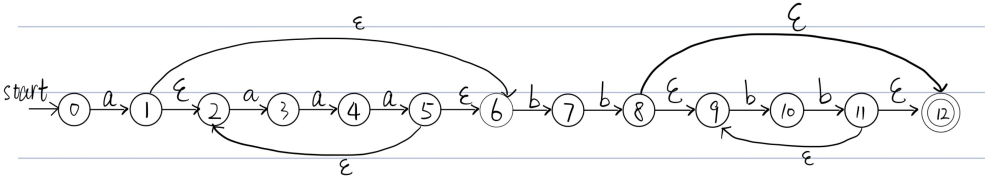
都不匹配

3. Let $\{a^{3n+1}b^{2m}|n \geq 0, m \geq 1\}$ denotes the set of all strings of a's and b's which start with $3n+1$ a's, followed by $2m$ b's.

1. **Please write the regular expression for these strings.**
2. **Convert the regular expression into NFA.**
3. **Convert the NFA to DFA.**

1. $a(aaa)^*bb(bb)^*$

2.

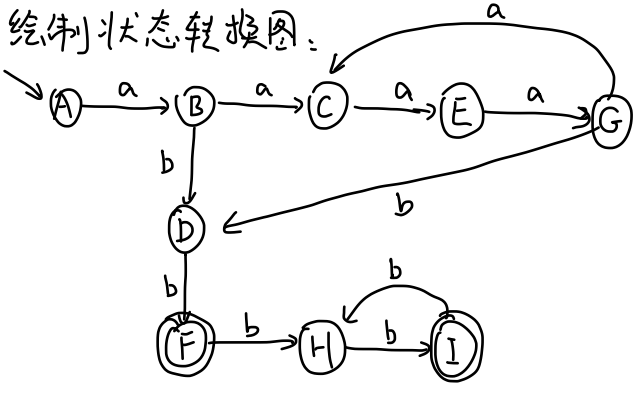


3. NFA确定化，构建状态转换表

init: ε-closure(0) = {0}

NFA	DFA	a	b
0	A	B	\
1, 2, 6	B	C	D
3	C	E	\
7	D	\	F
4	E	G	\
8, 9, 12	F	\	H
2, 5, 6	G	C	D
10	H	\	I
9, 11, 12	I	\	H

绘制状态转换图：



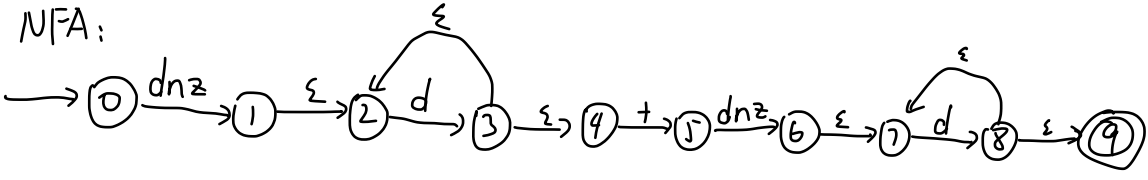
4. Use regular expression to describe additive expression of two integers, such as: 123+40, 7+55..... Draw the NFAs of these regular expressions.

digit = [0-9]

digit_no_zero = [1-9]

加法表达式： digit_no_zero (digit)* + digit_no_zero (digit)*

NFA：



5. Consider the grammar G[S]: S→Ac|aB; A→ab; B→bc. Is this grammar ambiguous? If so, please give your reason.

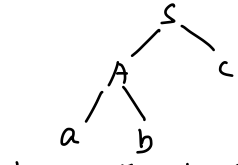
S → Ac

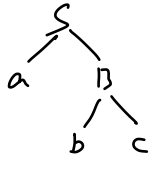
S → aB

A → ab

B → bc

对于语句 abc 构造语法树：





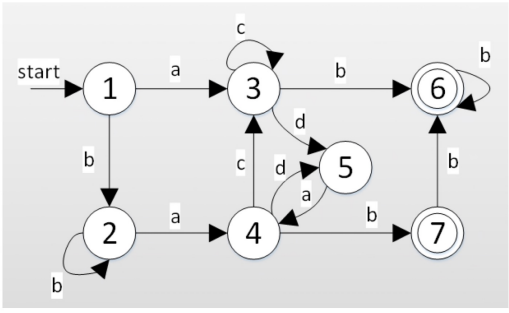
有两种语法树，因具有二义性

6. Given a context free grammar G[S], construct the parsing table for LL(1) parsing. G[S]: S→AB; B→S|ε; A→aC; C→aC|ε.

	First	Follow
S	a	$
A	a	a, $
B	a, ε	$
C	a, ε	a, $

	a	$
S	S→AB	
A	A→aC	
B	B→S	B→ε
C	C→aC C→ε	C→ε

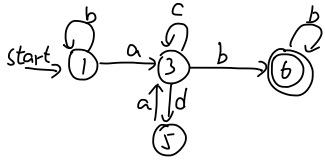
7. Please minimize the DFA shown in the following figure.



初始状态：{1,2,3,4,5}，{6,7}

① {1,2,5}，{3,4}，{6,7}

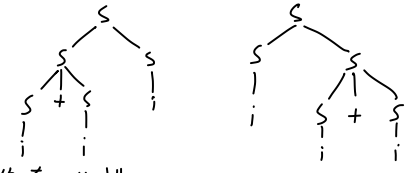
②



8. Given a context free grammar G[S]: S→S+S; S→S*S; S→(S); S→i. Is the grammar ambiguous?

S→S+S
S→S*S
S→(S)
S→i

对于文法 i+i+i：



故有二义性

9. Consider the following grammar: S→S∨T|T; T→T∧F|F; F→¬F|p|q.
Please write the rightmost derivation for the sentence form "¬F∨¬q∧p".
And give the handle of this sentence form.

$S \Rightarrow S \vee T$ 句柄为：$\neg F$

$\Rightarrow S \vee T \wedge F$

$\Rightarrow S \vee T \wedge p$

$\Rightarrow S \vee F \wedge p$

$\Rightarrow S \vee \neg F \wedge p$

$\Rightarrow S \vee \neg q \wedge p$

$\Rightarrow T \vee \neg q \wedge p$

$\Rightarrow F \vee \neg q \wedge p$

$\Rightarrow \neg F \vee \neg q \wedge p$

10. Consider the following grammar G[S]: S→(L)|aS|a ; L→L,S|S . Please rewrite this grammar to left factor and eliminate left recursion.

$S \to (L) \mid aS \mid a$

提取左公因子：

$S \to (L) \mid aS'$

$S' \to S \mid \varepsilon$

$L \to L,S \mid S$

消除左递归：

$L \to SL'$

$L' \to ,SL' \mid \varepsilon$

可得文法为：

$S \to (L) \mid aS'$

$S' \to S \mid \varepsilon$

$L \to SL'$

$L' \to ,SL' \mid \varepsilon$

11. Consider the following augmented SLR(1) grammar G'[S']: (0)S'→S ; (1)S→SS+ ; (2)S→SS* ; (3)S→a , and its SLR(1) parsing table:

	Action				Goto
	a	+	*	$	S
0	s2				1
1	s2			acc	3
2	r3	r3	r3	r3	
3	s2	s4	s5		3
4	r1	r1	r1	r1	
5	r2	r2	r2	r2	

Show the parsing stack and action of SLR(1) parser for the input "aaa+*".

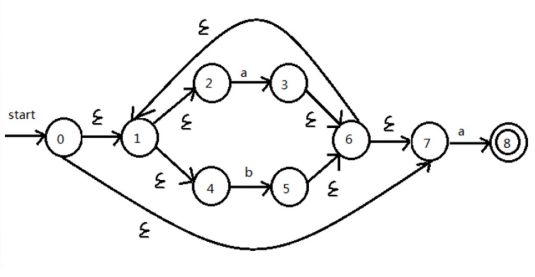
步骤	状态	栈	输入	动作	下一状态
1	0	$	aaa+*$	S_2	2
2	02	$a	aa+*$	r_3	1
3	01	$S	aa+*$	S_2	2
4	012	$Sa	a+*$	r_3	3
5	013	$SS	a+*$	S_2	2
6	0132	$SSa	+*$	r_3	3
7	0133	$SSS	+*$	S_4	4
8	01334	$SSS+	*$	r_1	3
9	013	$SS	*$	S_5	5
10	0135	$SS*	$	r_2	1
11	01	$S	$	acc	

12. Given a context free grammar G[S]: S→aAcBe ; A→b|Ab ; B→d , please write a rightmost derivation for string "abbcde", and give the viable prefixes for the string.

S → aAcBe
→ aAcde
→ aAbcde
→ abbcde

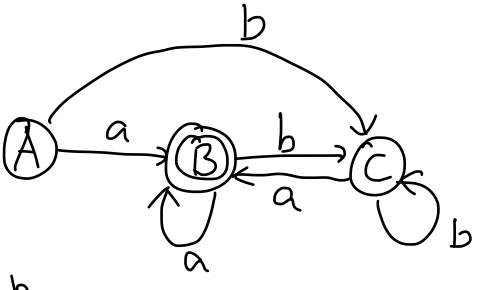
活前缀为：
{a, ab, aA, aAb, aAc, aAcd}

13. Construct the minimus-state DFA for the following NFA:



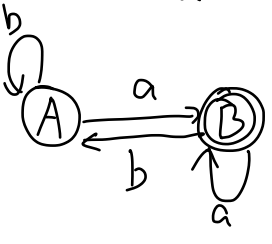
1. **Convert the NFA to DFA by subset construction. Both the transition table and the transition graph of DFA are required.**
2. **Minimize the states of the DFA.**

NFA	DFA	a	b
0,1,2,4,7	A	B	C
1,2,3,4,6,7,8	B	B	C
1,2,4,5,6,7	C	B	C



第1个化DFA：

① {A, C}, {B}



14. Rewrite the grammar G[A]: A→aABe|a ; B→Bb|d . Judge the grammar is LL(1) or not.

$A \to aABe \mid a$　　$B \to Bb \mid d$

提取左公因子：　　消除左递归：

$A \to aA'$　　$B \to dB'$

$A' \to ABe \mid \varepsilon$　　$B' \to bB' \mid \varepsilon$

判断是否 LL(1) 文法：

$First(ABe) = a$
$Follow(A') = d, \$$　不相交

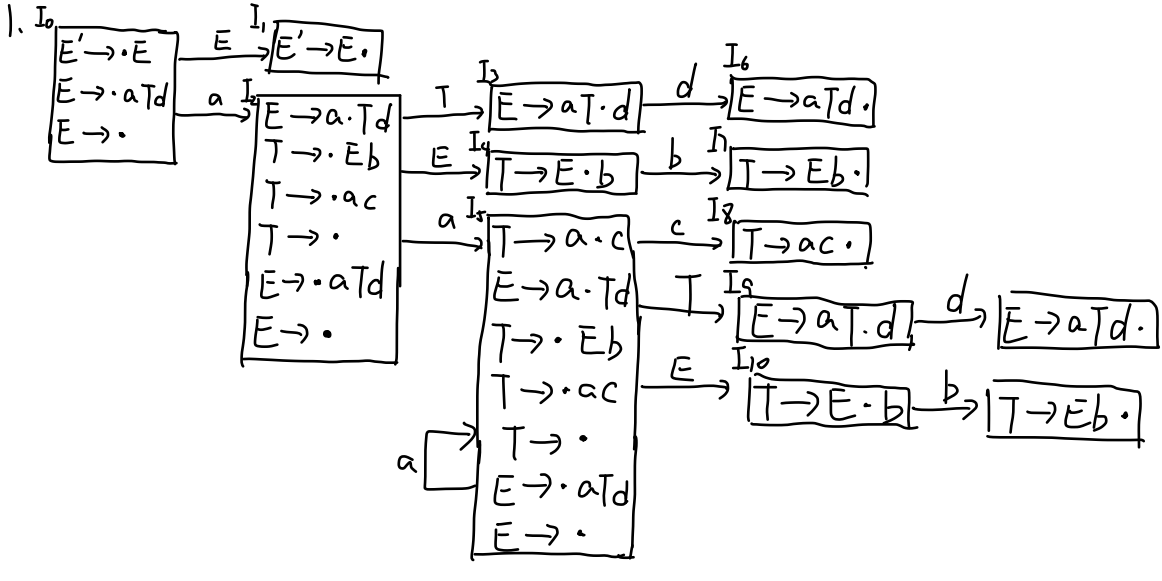
$First(bB') = b$
$Follow(B') = e$　不相交

因文法为 LL(1) 文法

15. Consider the following augmented grammar G'[E']: (1)E'→E (2)E→aTd (3)E→ε (4)T→Eb (5)T→ac (6)T→ε

1. **Construct the DFA of LR(0) items for this augmented grammar.**
2. **Construct the SLR(1) parsing table. Is this grammar SLR(1) grammar?**

1、



2、$Follow(E) = \{b, \$\}$ 和 $\{a\}$ 不相交

$Follow(T) = \{d\}$ 和 $\{a, First(T) = \{a, \varepsilon\}, First(E) = \{a, c\}\}$ 不相交

故为 SLR(1) 文法。

16. Consider the grammar G[S]: S→i|(E) ; E→SE' ; E'→+SE'|-SE'|ε

1. **Compute the FIRST SET and FOLLOW SET for the non-terminals.**
2. **Is the grammar is LL(1)?**
3. **Construct the LL(1) parsing table.**

1.

	First	Follow
S	i, (	$, +, -,)
E	i, (	)
E'	+, -, ε	)

2、First(i) ∩ First((E)) = ϕ

First(+SE') ∩ First(-SE') = ϕ

因为E'→ε,

Follow(E') =) 和 First(+SE') ∩ First(-SE')不相交

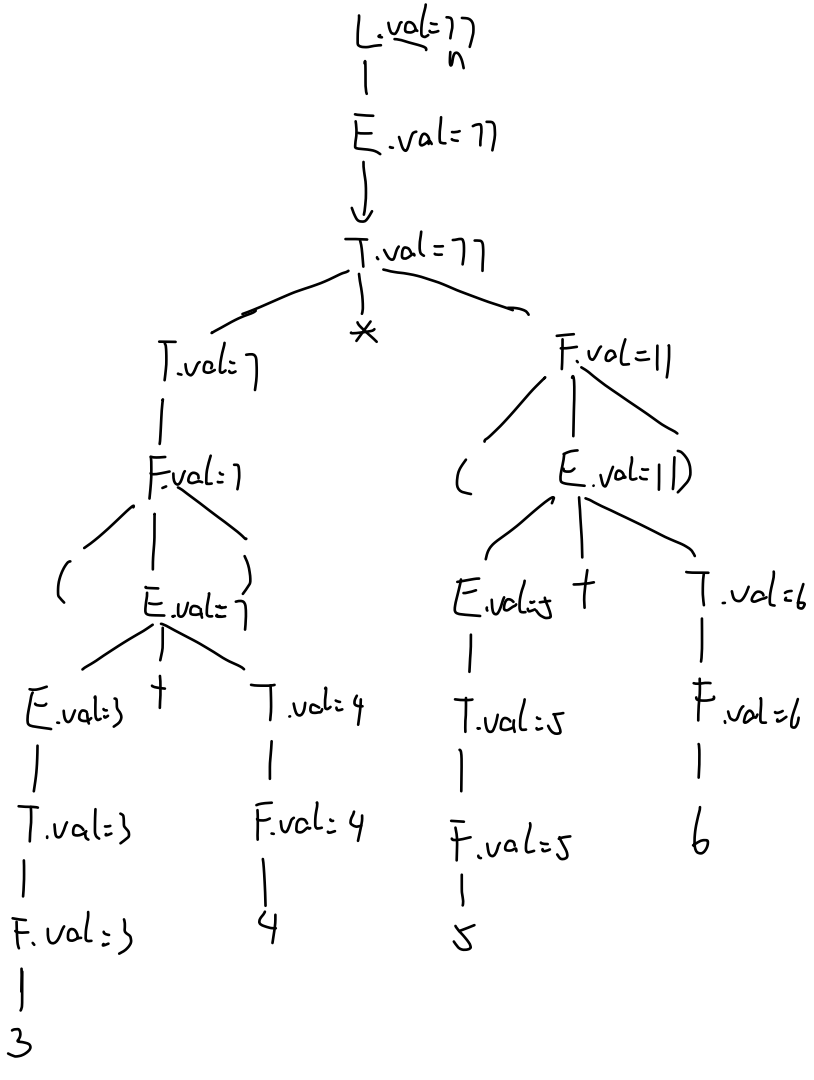
故为LL(1)文法

3.

	i	(	)	+	-	$
S	S→i	S→(E)				
E	E→SE'	E→SE'				
E'			E→ε	E'→+SE'	E'→-SE'	

17. For the following SDD, give annotated parsint tree for the expression (3+4)*(5+6)n.

产生式	语义规则
1) $L \to E$ n	$L.val = E.val$
2) $E \to E_1 + T$	$E.val = E_1.val + T.val$
3) $E \to T$	$E.val = T.val$
4) $T \to T_1 * F$	$T.val = T_1.val \times F.val$
5) $T \to F$	$T.val = F.val$
6) $F \to (E)$	$F.val = E.val$
7) $F \to$ **digit**	$F.val =$ **digit**.lexval



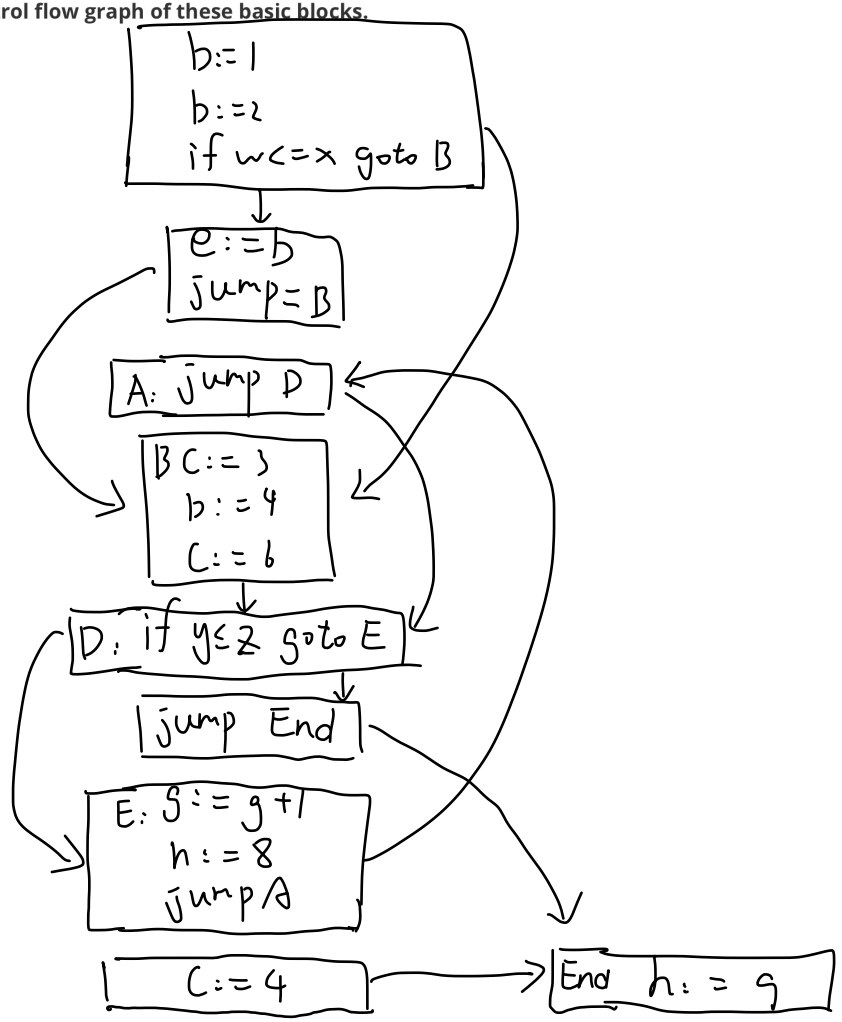
18. Consider the following fragment of three-address instructions.

```
→(1)              b := 1
 (2)              b := 2
 (3)              if w <= x goto B
→(4)              e := b
 (5)              jump B
→(6)      A:      jump D
→(7)      B:      c := 3
 (8)              b := 4
 (9)              c := 6
→(10)     D:      if y <= z goto E
→(11)             jump End
→(12)     E:      g := g + 1
 (13)             h: = 8
 (14)             jump A
→(15)             c:=4
→(16)     End:    h := 9
```

Please partition these three-address instructions into basic blocks:

1. **Find the leaders of these basic blocks.**
2. **Draw the control flow graph of these basic blocks.**

(1)

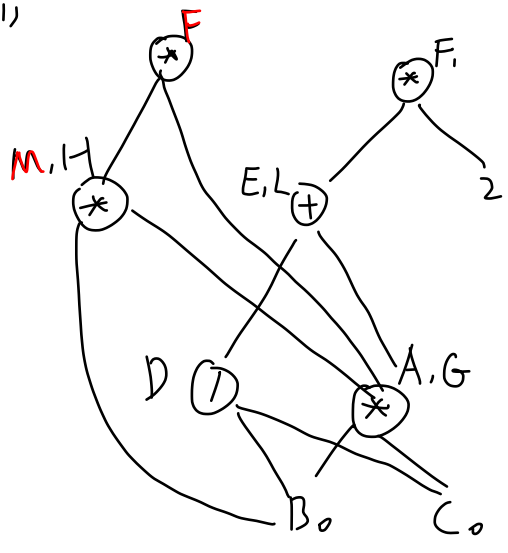


19. Consider the following basic block:

```
A:=B*C  D:=B/C  E:=A+D  F:=E*2  G:=B*C
H:=G*B  F:=H*G  L:=E    M:=H
```

1. **Construct a DAG for this basic block.**
2. **Assuming that only F and M are live on exit from this basic block, simplify the three-address code.**

(1)



(2) 只有F和M在终点后活跃

①删除无活跃变量根结点，如：F1，2，E，L，D

②对于局部公共子表达式，只保留一个 M，A

③优化后代码为：

A = B * C

M = A * B

F = M * A

20. The following grammar generates binary strings and their complements (10 points).

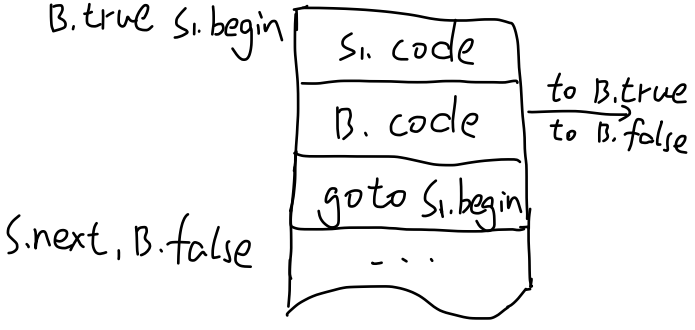
$F \rightarrow B \mid \neg B$

$B \rightarrow B0 \mid B1 \mid 0 \mid 1$

The value of a (non-negated) string is just the decimal value of the binary number the string represents; the value of a negated string is the decimal value of the string with 1′ s replaced by 0′ s and 0′ s replaced by 1′ s. For example, the value of 010 is 2 and ¬010 is 5. Design a syntax-directed definition (SDD) for the above grammar such that the non-terminal F has an attribute F.val which keeps the value of an input string generated by F. Please do NOT modify the grammar.

产生式	语义规则
$F \rightarrow B$	$F.val = B.val$
$F \rightarrow \neg B$	$F.val = 2^{B.len} - B.val - 1$
$B \rightarrow B0$	$B.val = B.val \times 2$; $B.len++$
$B \rightarrow B1$	$B.val = B.val \times 2 + 1$; $B.len++$
$B \rightarrow 0$	$B.val = 0$; $B.len = 1$
$B \rightarrow 1$	$B.val = 1$ $B.len = 1$

21. Write SDD to generate the three-address code for *"Do statement while(logical-expression)".*



产生式

$S \rightarrow$ do S_1 while B

语义规则

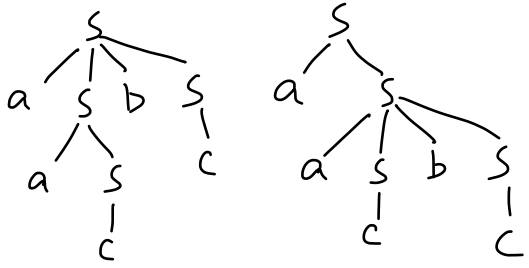
```
S1.begin = newlabel();
B.true = newlabel();
B.false = S.next;
S.code = label(S1.begin) ||
         label(B.true) ||
         S1.code || B.code ||
         gen('goto', S1.begin);
```

1. **Consider the following grammar G[S]:**

   S→aSbS | aS | c

   Is this grammar ambiguous? if so, please give your reason with an example string and its parse trees.

对于串 aacbc 存在以下两种语法树：

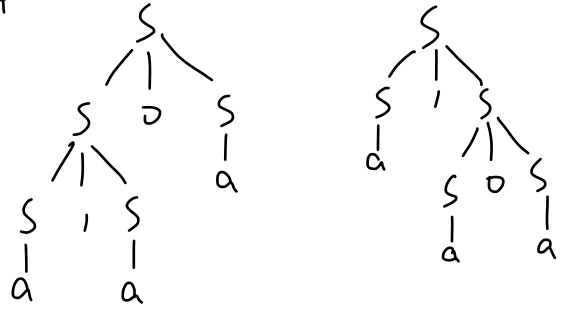


2. **Consider the following grammar G[S]:**

   S → S0S | S1S | a

   Is this grammar ambiguous? if so, please give your reason with an example string and its parse trees.

对于串a1a0a 存在两棵语法树：



3. **Consider the following grammar :**

G[S]: S→(L) | aS | a

L→L,S | S

Please write the rightmost derivation for the sentential form '(S, (a))', and give the handle and the viable prefixes of this sentential form.

$S \to (L)$

$\to (L,S)$

$\to (L,(L))$

$\to (L,(S))$

$\to (L,(a))$

$\to (S,(a))$

句柄为：S

可行前缀为：{ε, (, (S}

4. **Consider the following grammar :**

G[S]: S→aAcB | Bd

A→AaB | c

B→bScA | b

Please write the rightmost derivation for the sentential form 'aAcbBdcc', and give the handle and the viable prefixes of this sentential form.

$S \to aAcB$

$\to aAcbScA$

$\to aAcbScc$

$\to aAcbBdcc$

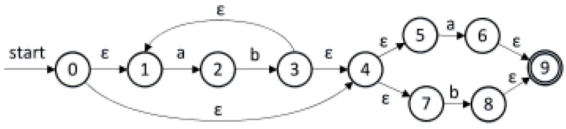
可行前缀为：

{a, aA, aAc, aAcb, aAcbB, aAcbBd, ε}

句柄为：Bd

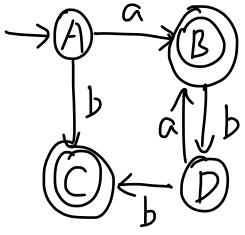
5. Construct the minimum-state DFA for the following NFA:

1) **Convert this NFA into DFA by subset construction. Both the transition table and the transition graph of DFA are required.**
2) **Minimize the states of this DFA.**



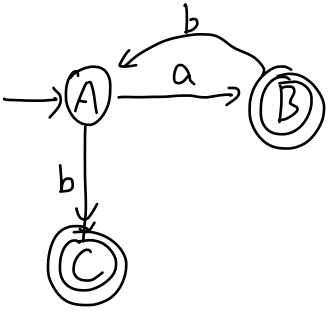
(1)

NFA	DFA	a	b
0,1,4,5,7	A	B	C
2,6,9	B		D
8,9	C		
1,3,4,5,7	D	B	C



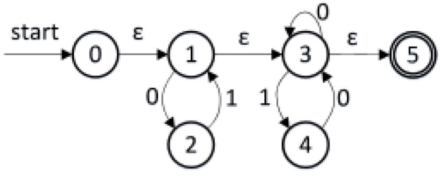
(2) {A, D}, {B, C}

{A, D}, {B}, {C}



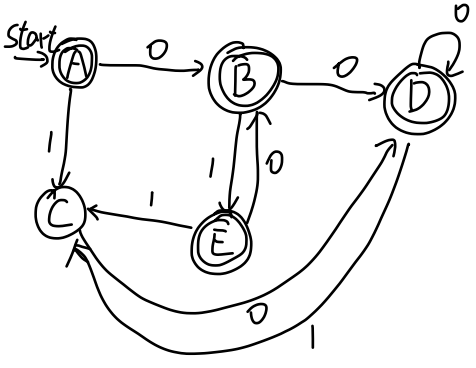
6. Construct the minimum-state DFA for the following NFA:

1) **Convert this NFA into DFA by subset construction. Both the transition table and the transition graph of DFA are required.**
2) **Minimize the states of this DFA.**



(1)

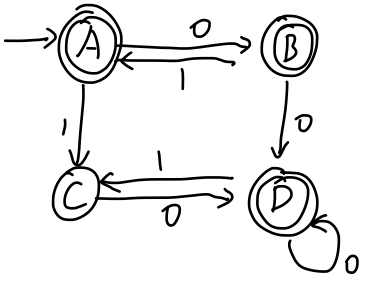
NFA	DFA	0	1
0,1,3,5	A	B	C
2,3,5	B	D	E
4	C	D	
35	D	D	C
1345	E	B	C



(2)

{C}, {A, B, D, E}

{C}, {A, E}, {B}, {D}



7. Consider the following grammar G[S]:

S→ k | (T)

T→ T * S | T / S | S

(1) Please rewrite this grammar to eliminate left recursion.

(2) Compute FIRST and FOLLOW for the grammar. Please explain the rewritten grammar is LL(1) grammar or not.

(3) Construct the parsing table

1. $T \to T*S \mid T/S \mid S$ （α = $*S$ 或 $/S$，β = S）消除左递归为：

$S \to k \mid (T)$

$T \to ST'$

$T' \to *ST' \mid /ST' \mid \varepsilon$

2.

	First	Follow
S	k, (	$, *, /,)
T	k, (	)
T'	*, /, ε	)

判断是否为LL(1)文法：

① First(k) ∩ First((T)) = ∅

② First(*ST') ∩ First(/ST') = ∅

因为 T' → ε

③ Follow(T') ∩ First(*ST') = ∅

Follow(T') ∩ First(/ST') = ∅

故该文法为LL(1)文法。

3.

	k	(	)	*	/	$
S	S→k	S→(T)				
T	T→ST'	T→ST'				
T'			T'→ε	T'→*ST'	T'→/ST'	

8. Consider the grammar (decls, decl, type, varlist and varlist' are non-terminals):

decls → decl;decls | ε (1, 2)　　decl → type varlist (7)

type → int | bool (3, 4)　　varlist → id varlist' (8)

varlist' → , varlist | ε (5, 6)

1) **Construct First and Follow sets for the nonterminals.**
2) **Is this grammar the LL(1) grammar? Give your reason.**
3) **Construct the LL(1) parsing table.**

(1)

	First	Follow
decls	int, bool, ε	$
decl	int, bool	;
type	int, bool	id
varlist	id	;
varlist'	, ε	;

(2) Follow(decls) ∩ First(decl) = ϕ
First(int) ∩ First(bool) = ϕ
Follow(varlist') ∩ First(,) = ϕ

故文法為LL(1)文法

(2)

	int	bool	;	,	id	$
decls	1	1				2
decl	7	7				
type	3	4				
varlist					8	
varlist'			6	5		

9. Given the grammar G[S]: S→(S)A | aA , A→ BA | ε , B→ S | +S | *. (here, A→BA having higher priority on A→ ε), and the parsing table of G[S] as follow.

	a	(	)	+	*	$
S	S→aA	S→(S)A				
A	A→ ε A→BA	A→ ε A→BA	A→ ε	A→ ε A→BA	A→ ε A→BA	A→ ε
B	B→S	B→S		B→+S	B→*	

To achieve the predictive parsing, the rule "A→BA having higher priority on A→ε when selecting a production of A for derivations" is added into the grammar G[S]. Please give the parsing process for the input string a*a in the following table.

step	stack	input	action
1	S$	a*a$	
2	aA$	a*a$	S→aA
3	A$	*a$	匹配a
4	BA$	*a$	A→BA
5	*A$	*a$	B→*
6	A$	a$	匹配*
7	BA$	a$	A→BA
8	SA$	a$	B→S
9	aAA$	a$	S→aA
10	AA$	$	匹配a
11	A$	$	A→ε
12	$	$	A→ε

10.Given the grammar G[S]: S→(S)A | aA , A→ BA | ε , B→ S | +S | *. (here, A→BA having higher priority on A→ ε), and the parsing table of G[S] as follow.

	a	(	)	+	*	$
S	S→aA	S→(S)A				
A	A→ ε A→BA	A→ ε A→BA	A→ ε	A→ ε A→BA	A→ ε A→BA	A→ ε
B	B→S	B→S		B→+S	B→*	

To achieve the predictive parsing, the rule "A→BA having higher priority on A→ε when selecting a production of A for derivations" is added into the grammar G[S]. Please give the parsing process for the input string (a*a) in the following table.

Step	stack	input	action
1	S$	(a*a)$	
2	(S)A$	(a*a)$	S→(S)A
3	S)A$	a*a)$	match (
4	aA)A$	a*a)$	S→ aA
5	A)A$	*a)$	match a
6	BA)A$	*a)$	A→BA
7	*A)A$	*a)$	B→*
8	A)A$	a)$	match *
9	BA)A$	a)$	A→ BA
10	SA)A$	a)$	B→S
11	aAA)A$	a)$	S→aA
12	AA)A$	)$	match a
13	A)A$	)$	A→ ε
14	)A$	)$	A→ ε
15	A$	$	match)
16	$	$	A→ ε

11.Consider the following augmented grammar G[S']:

(0) S'→S (1) S→Pa (2) S→Pb (3) S→c

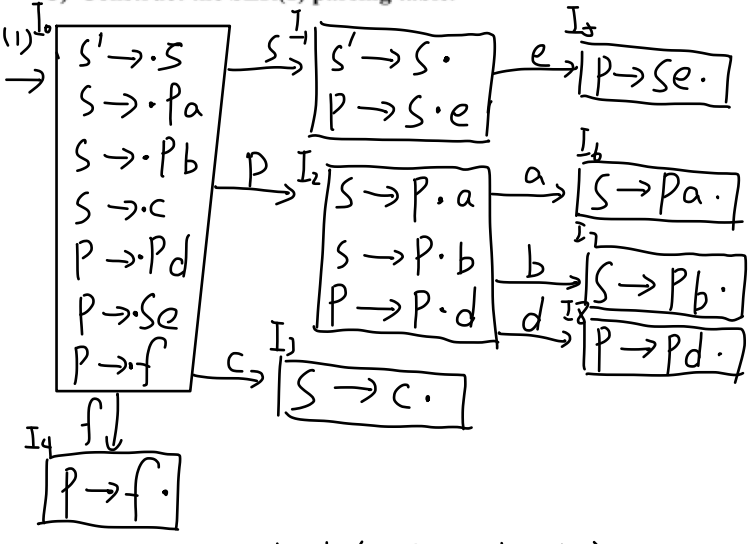
(4) P→Pd (5) P→Se (6) P→f

1) Construct the DFA of LR(0) items for this augmented grammar.

2) Is this grammar the LR(0) or SLR(1) grammar ? Give your reason.

3) Construct the SLR(1) parsing table.

(1)



(2) 无移入归约冲突因为 LR(0) 也为 SLR(1)

(3)

	a	b	c	d	e	f	$	S	P
0			s3			s4		1	2
1					s5		acc		
2	s6	s7		s8					
3					r3		r3		
4	r6	r6		r6					
5	r5	r5		r5					
6					r1		r1		
7					r2		r2		
8	r4	r4		r4					

12. Consider the following augmented grammar G[S']:

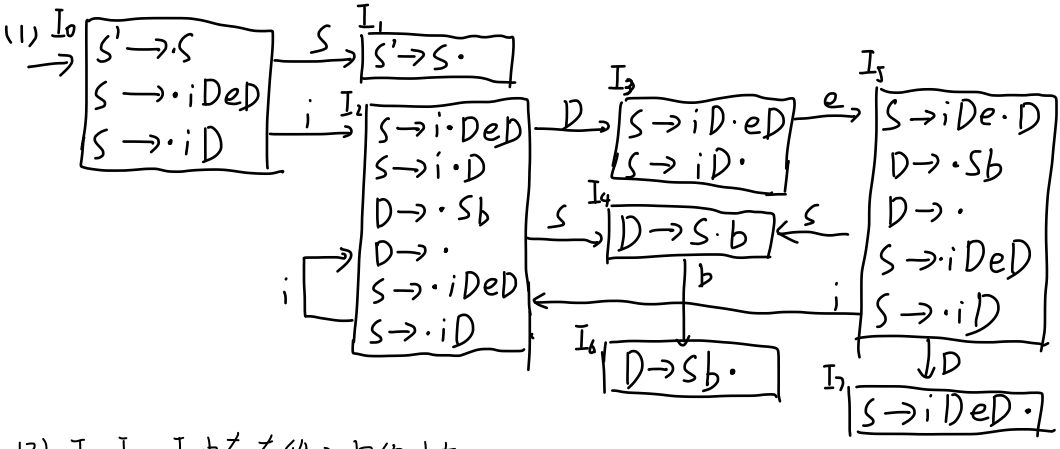
(0) S'→S (1) S→iDeD (2) S→ iD (3) D→Sb (4) D→ε

1) Construct the DFA of LR(0) items for this augmented grammar.

2) Is this grammar the LR(0) or SLR(1) grammar ? Give your reason.

3) Construct the SLR(1) parsing table.

(1)



(2) I_2、I_3、I_5中存在移入归约冲突，不为LR(0)

	First	Follow
S'	i	$
S	i	$, b
D	i, ε	e, b, $

Follow(D) ∩ {i} = ϕ

Follow(S) ∩ {e} = ϕ

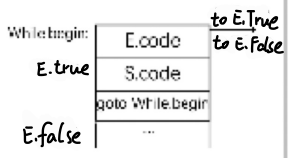
Follow(D) ∩ {i} = ϕ

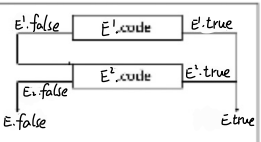
故为SLR(1)文法

(3)

	i	e	b	$	S	D
0	s2				1	
1				acc		
2	s2	r4	r4	r4	4	3
3		s5	r2	r2		
4			s6			
5	s2	r4	r4	r4	4	7
6		r3	r3	r3		
7			r1	r1		

13. Consider the following attribute grammar:

Grammar	Semantic Rules	
While-stmt→ while E do S	While-stmt.begin=newlabel; While-stmt.next=newlabel; E.true=newlabel; E.false= While-stmt.next; S.next= While-stmt.begin; While-stmt.code=Label While-stmt.begin || E.code || Label E.true || S.code ||goto While-stmt.begin||Label While-stmt.next	

E → E1 or E2	E1.true=E.true; E1.false=newlabel; E2.true=E.true; E2.false=E.false; E.code=E1.code || Label E1.false || E2.code	
E → id1 relop id2	E.code = if id1.name relop id2.name goto E.true || goto E.false	
S → id = id+num	S.code = id.name = id.name + num.val	

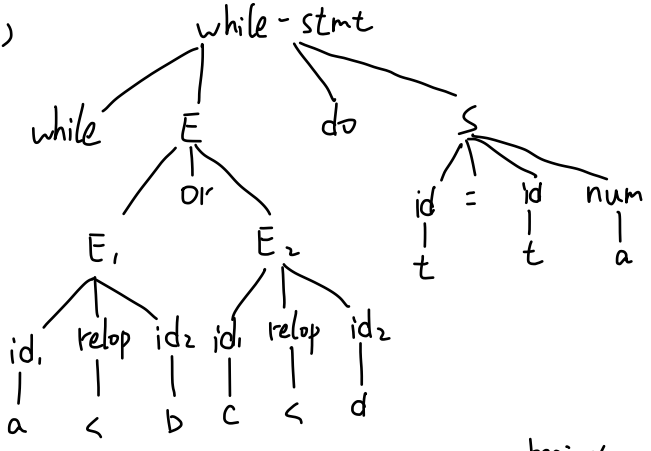
Given the source code: while a<b or c<d do t= t+a

(1) Draw the Abstract Syntax Tree

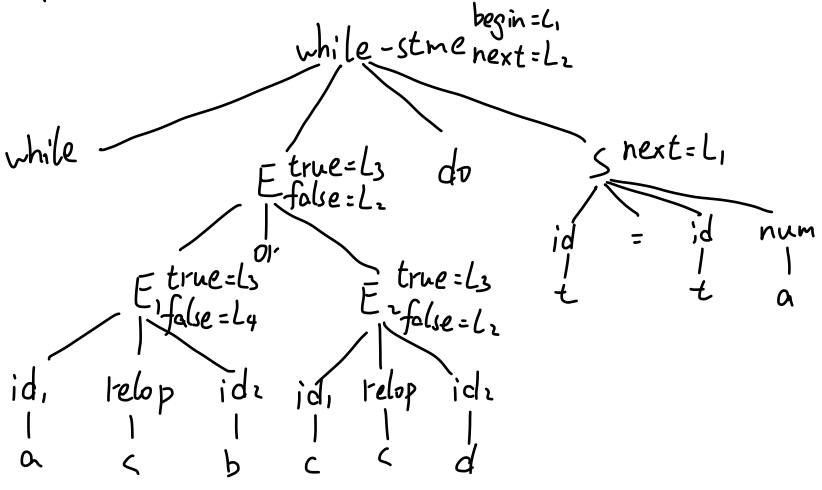
(2) According to the semantic rules, calculate the inherited attributes 'true' ,'false' and 'next' on the corresponding nodes of the syntax tree, to form the semantic tree。

(3) Consider the step (2) result and the synthetic attribute 'code' , translate the three address code in a bottom-up order, recursively。

(1)



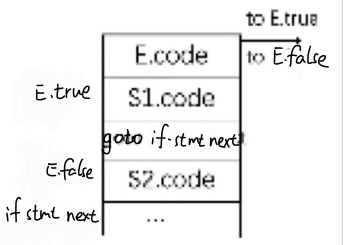
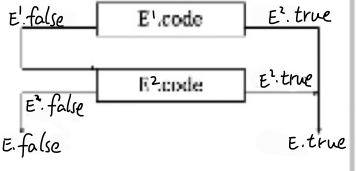
(2)



(3)

Label L_1
if $a < b$ goto L_3
goto L_4
Label L_4
if $c < d$ goto L_3
goto L_2
Label L_3
$t = t + a$
goto L_1
Label L_2

14. Consider the following attribute grammar:

Grammar	Semantic Rules	
if-stmt→ if E then S1 else S2	If-stmt.next=newlabel; E.true=newlabel; E.false=newlabel; S1.next=if-stmt.next; S2.next=if-stmt.next If-stmt.code=E.code || Label E.true || S1.code ||goto S.next || Label E.false || S2.code || if-stmt.next	
E → E1 or E2	E1.true=E.true; E1.false=newlabel; E2.true=E.true; E2.false=E.false; E.code=E1.code || Label E1.false || E2.code	
E → id1 relop id2	E.code = if id1.name relop id2.name goto E.true || goto E.false	
S → id = num	S.code = id.name = num.val	

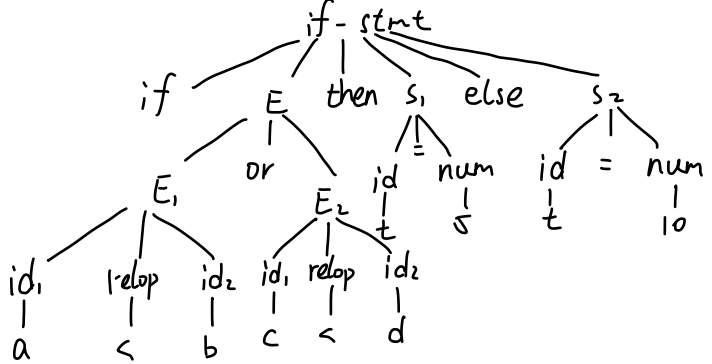
Given the source code: if a<b or c<d then t= 5 else t=10

(1) **Draw the Abstract Syntax Tree**

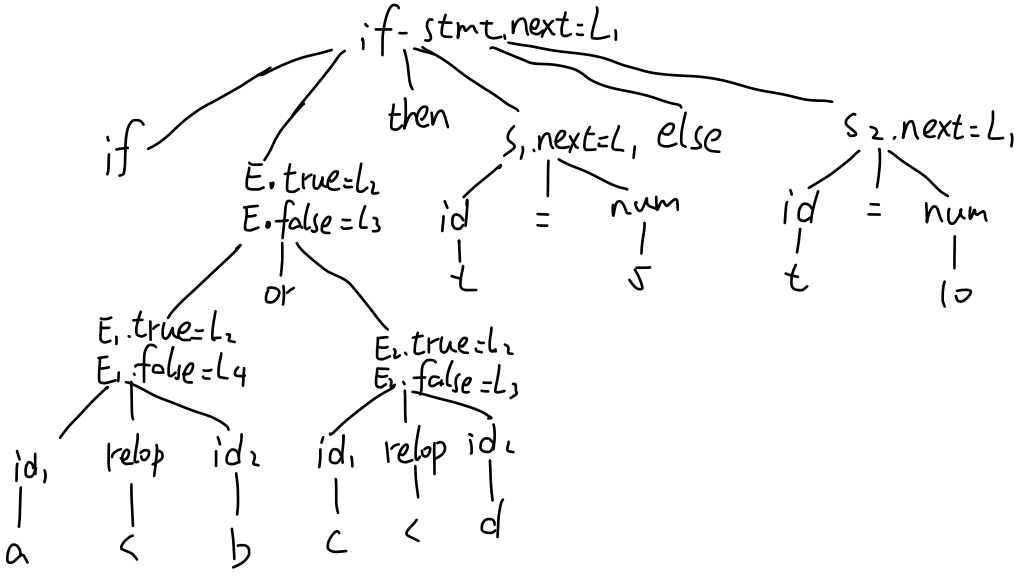
(2) **According to the semantic rules, calculate the inherited attributes 'true' ,'false' and 'next' on the corresponding nodes of the syntax tree, to form the semantic tree。**

(3) **Consider the step (2) result and the synthetic attribute 'code' , translate the three address code in a bottom- up order, recursively。**

(1) if a<b or c<d then t=5 else t=10



(2)



(3)

if a<b goto L2
goto L4
Label L4
if c<d goto L2
goto L3
Label L2
t=5
goto L1
Label L3
t=10
goto L1